



UNIVERSITAS
GADJAH MADA

Assignment 1

Single Layer Perceptron for Iris Binary Classification

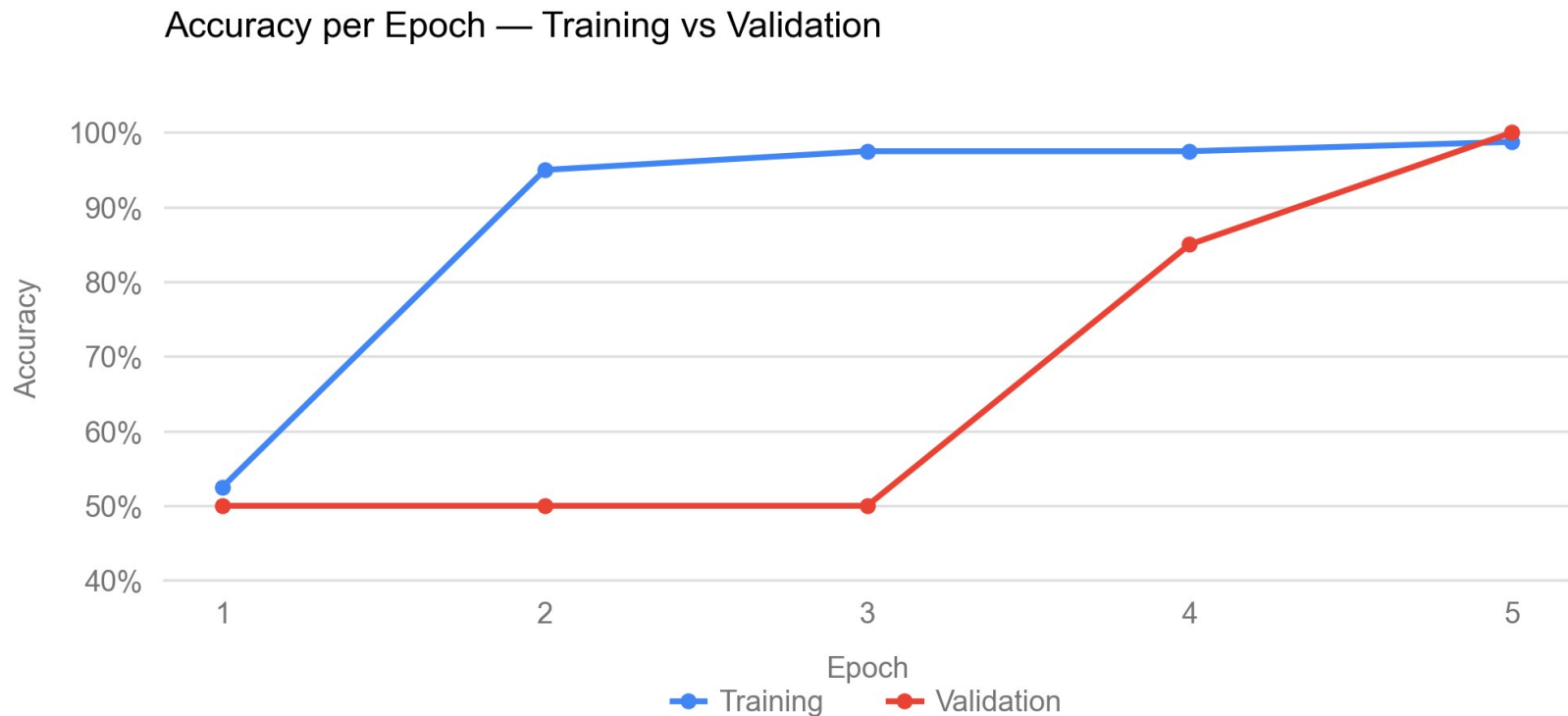
Muhammad Javier

24/546674/PA/23210

Deep Learning — Universitas Gadjah Mada

Accuracy — Google Sheets

Training vs validation accuracy per epoch, computed in the SLP spreadsheet (80 training / 20 validation samples).

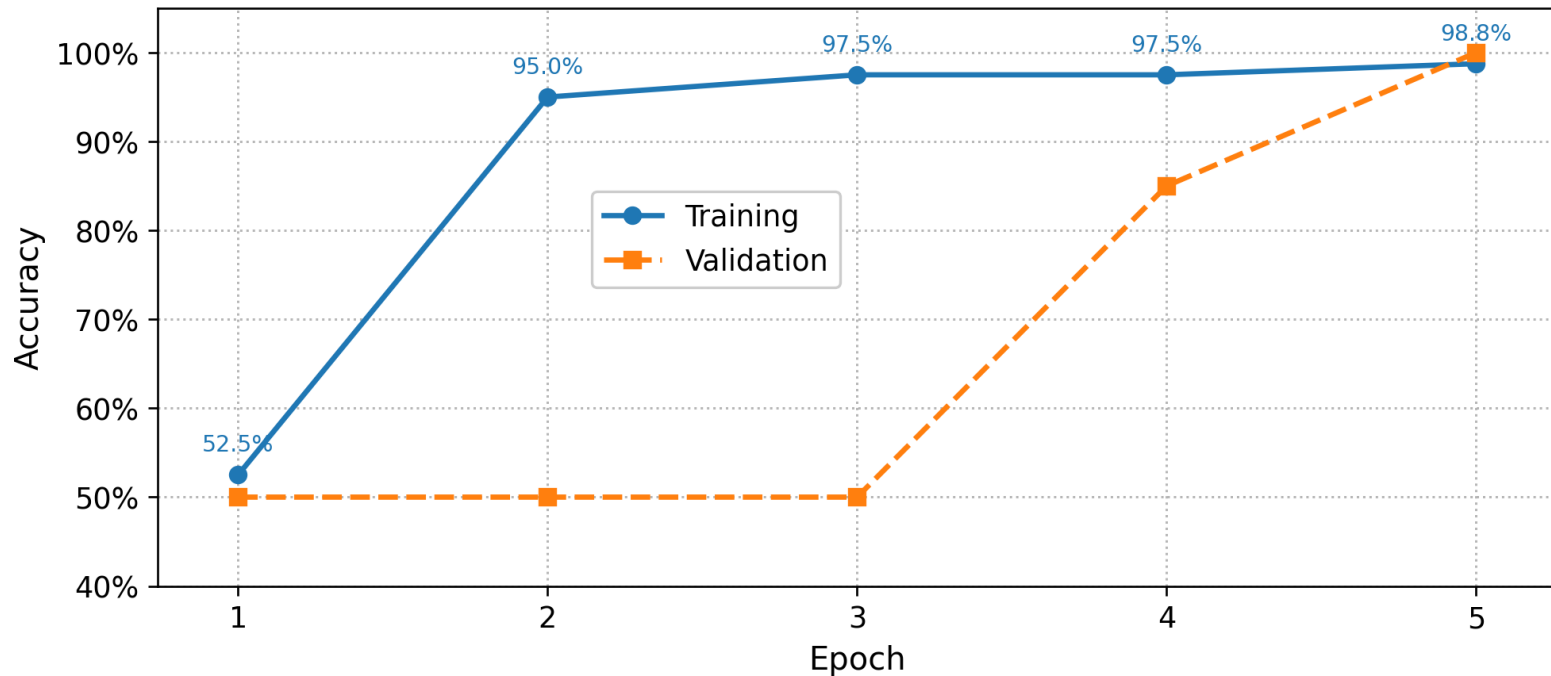


Training accuracy rises 52.50% → 98.75%. Validation sits at 50.00% while the boundary is still separating on sepal size, then reaches 85.00% at epoch 4 and 100.00% at epoch 5 as the petal weights take over.

Accuracy — Python Implementation

The same run reproduced by SLP-python.py — identical initialisation, update rule and sample order.

Accuracy per Epoch — Training vs Validation

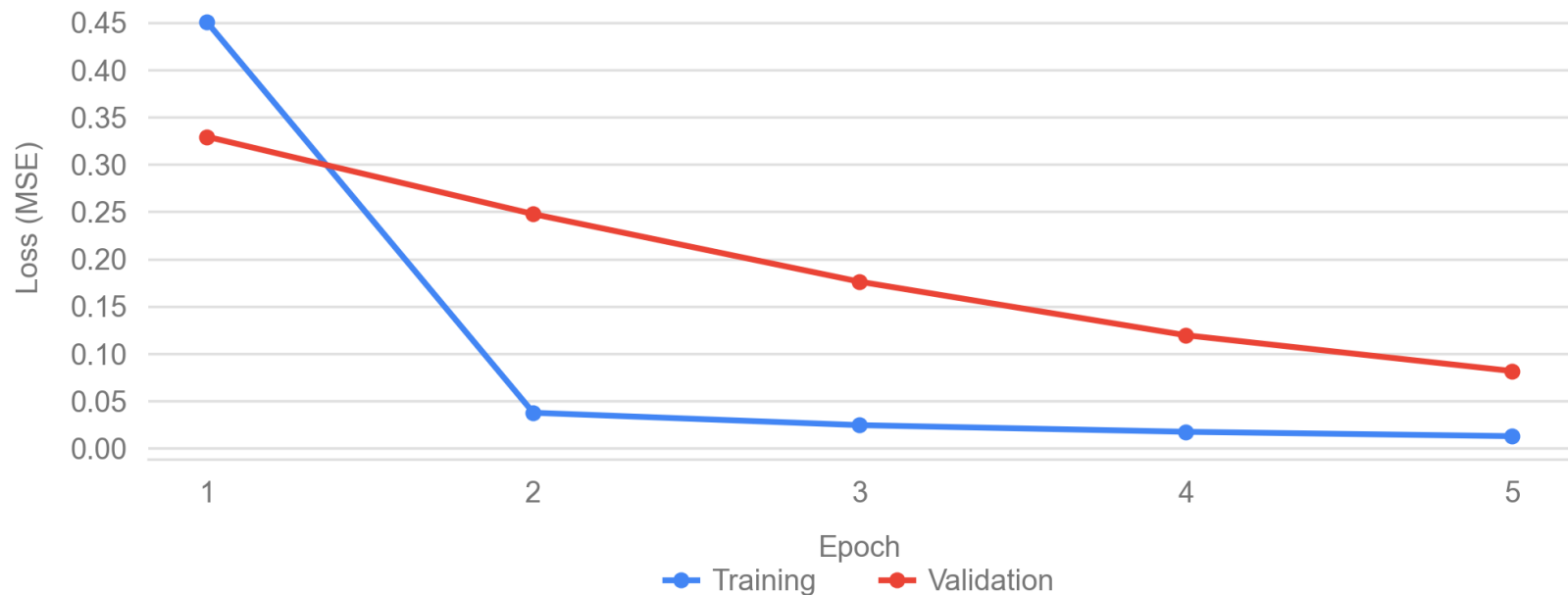


Every epoch matches the spreadsheet exactly: 52.50%, 95.00%, 97.50%, 97.50%, 98.75% (training) and 50.00%, 50.00%, 50.00%, 85.00%, 100.00% (validation).

Loss (MSE) — Google Sheets

Mean Sum-Square-Error per epoch. Validation reuses the bias and teta that training ends each epoch with, exactly as the worksheet specifies.

Loss (MSE) per Epoch — Training vs Validation

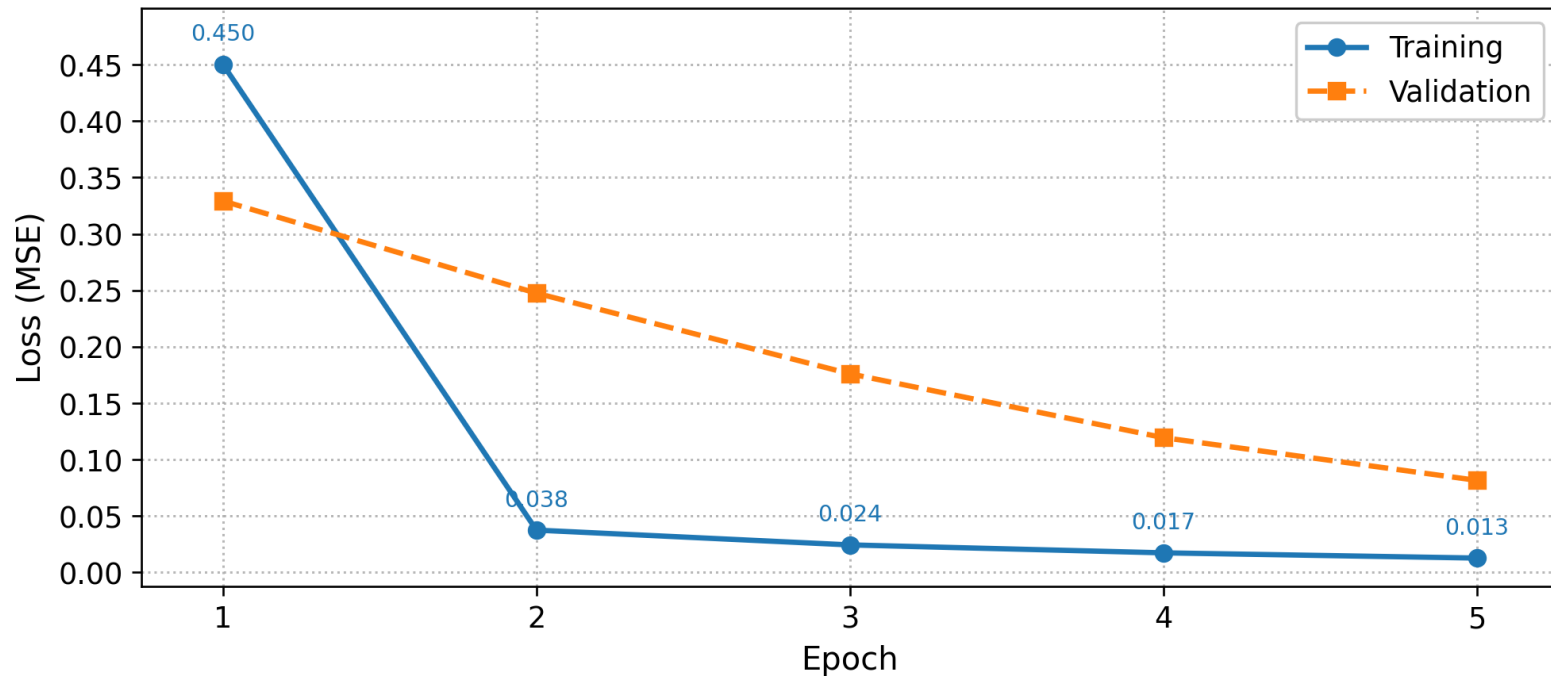


Training loss falls $0.450203 \rightarrow 0.012764$, the steepest drop in epoch 2. Validation loss decreases steadily $0.329163 \rightarrow 0.081583$ — it never turns upward, so there is no overfitting across these five epochs.

Loss (MSE) — Python Implementation

Loss curves produced by SLP-python.py for the same five epochs.

Loss (MSE) per Epoch — Training vs Validation



Training loss reproduces the spreadsheet's Sum-Square-Error column exactly — 0.4502033345, 0.0375290227, 0.0244179422, 0.0173886874, 0.0127640598 — confirming both implementations follow the same update rule.

Source Links

Spreadsheet and source code for Tasks 1 and 2.

01 Google Spreadsheet

Task 1 — SLP worksheet, 5 epochs, 80 training / 20 validation samples

<https://docs.google.com/spreadsheets/d/1zPx4ckdLXXAYSfJ87rMhDIB8mPoLJA7x/edit?usp=sharing&oid=118399330202018473068&rtpof=true&sd=true>

02 GitHub Repository

Task 2 — SLP-python.py, the Python implementation of the same model

<https://github.com/muhajav/SLP-Deep-Learning>

References

1. Afiahayati. (2026). Linear Classifier — Single Layer Perceptron [Lecture slides]. Department of Computer Science and Electronics, Universitas Gadjah Mada.
2. Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.
3. Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.
4. Dua, D., & Graff, C. (2019). Iris Data Set. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/53/iris>
5. UCI Machine Learning. (2016). Iris Species [Data set]. Kaggle. <https://www.kaggle.com/datasets/uciml/iris>
6. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
7. Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. · Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.